

Training of Image2Image translation

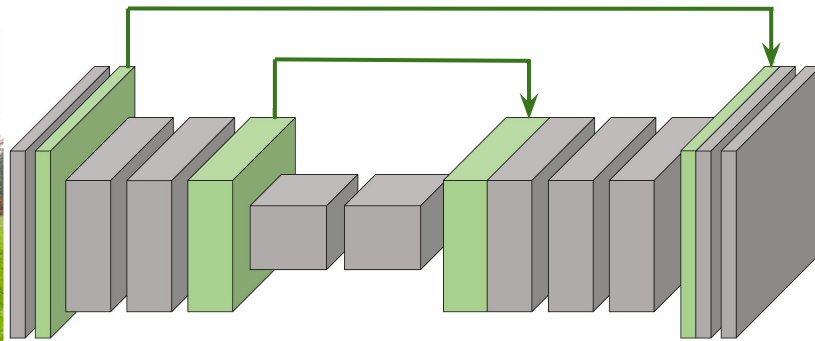
Input comes in (X,Y) pairs

Loss: squared loss or absolute loss

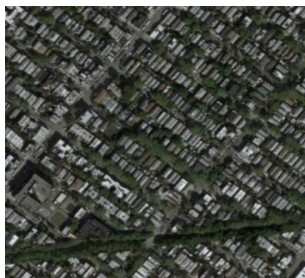
$$\mathcal{L}(G(X), Y) = |G(X) - Y|$$
$$\mathcal{L}(G(X), Y) = \|G(X) - Y\|^2$$

X

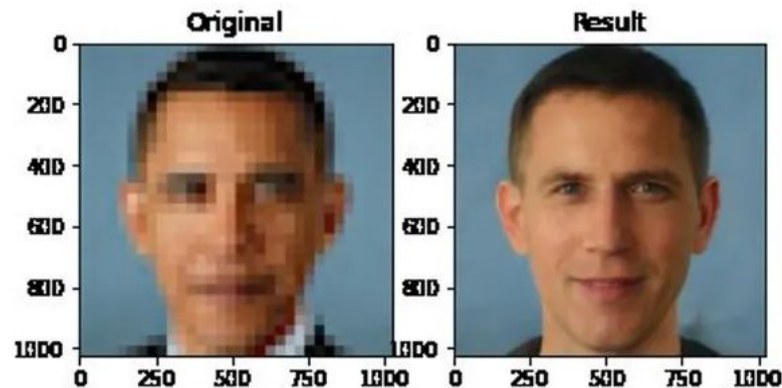
Y



Examples



But: Super-resolution inherits biases of training data. Filled in details are not real.



<https://www.theverge.com/21298762/face-depixer-ai-machine-learning-tool-pulse-stylegan-Obama-bias>

What happens if we don't have paired data?

Image Transformations with Deep Learning: Four Key Cases

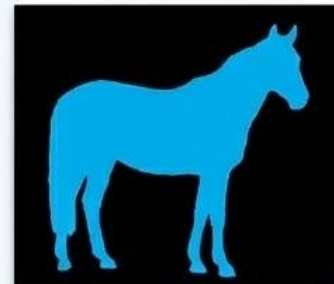
1. Image Classification



Output Label:
"Horse"

(Input: Image → Output: Label)

2. Paired Image Translation



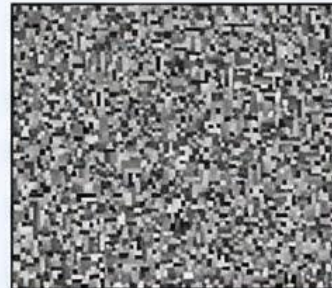
(Input: Image → Output: Image)

3. Unpaired Image Translation



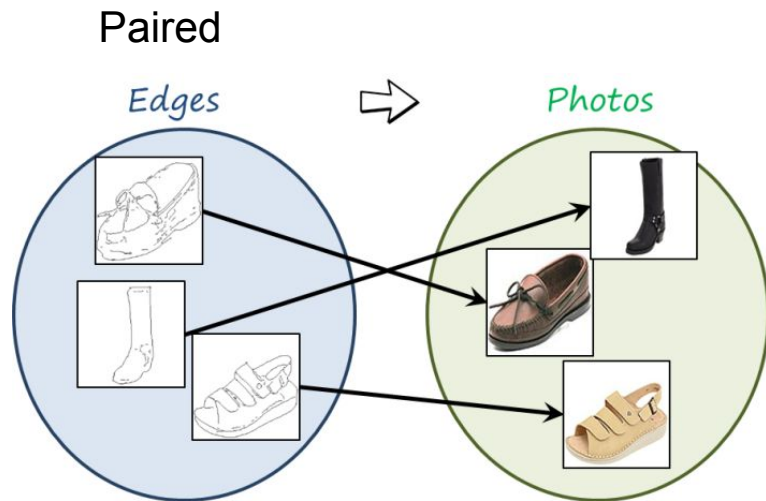
(Input: Image → Output: Image)

4. Image Generation



(Input: Noise → Output: Image)

Unpaired Image Translation. We have X but no Y!



[Generative adversarial networks and image-to-image translation | Luis Herranz](#)

Unpaired



[12 Astonishing Facts About Horses](#)



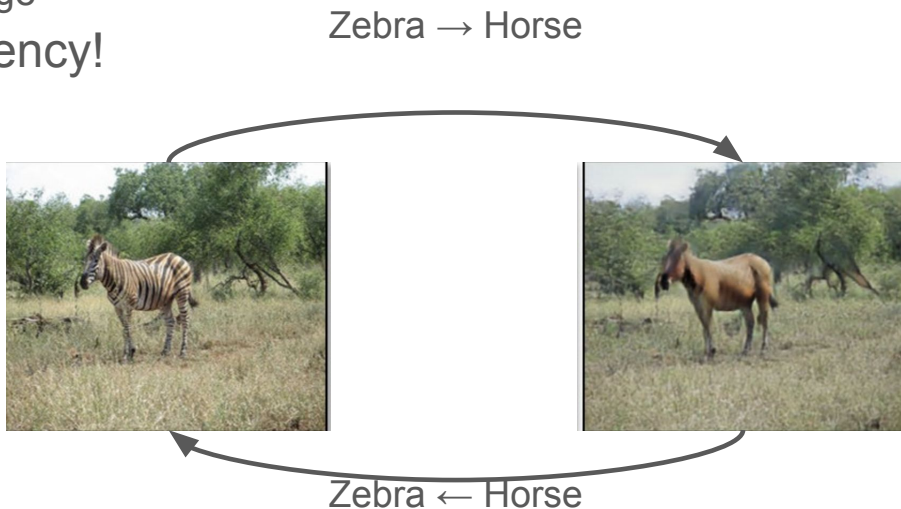
[Zebra Facts | Live Science](#)

Paired limitations - it is hard to find exact pairings

We have images of horses, we have images of zebras, but we don't have paired images of what a specific horse would look like as a zebra.

Key Idea: Cycle Consistency

- Image translation should be **invertible!**
 - Translating a zebra to a horse and then back to a zebra should recover the original image
- Cycle consistency!



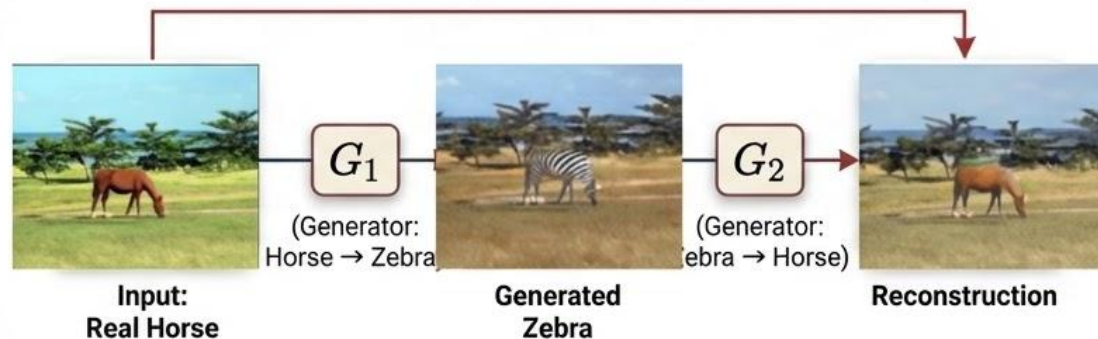


CycleGAN

Cycle Consistency & Reconstruction

$$\|\mathbf{x}_{\text{horse}} - G_2(G_1(\mathbf{x}_{\text{horse}}))\|_2^2$$

Reconstruction Loss: Squared error between original and reconstructed image, enforcing cycle consistency.



Q: The Cycle Consistency Loss is clever, but what trivial solution will you find if you minimize it alone?

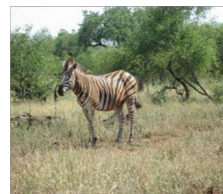
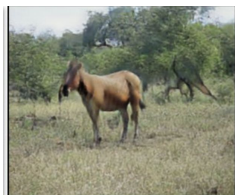
Translating a zebra to a horse and then back to a zebra should recover the original image

Classifier (Discriminator) to the rescue!

- A binary classifier that determines whether a given image is a zebra or not
- Can actually use as a learning signal!

Not Zebra

Zebra

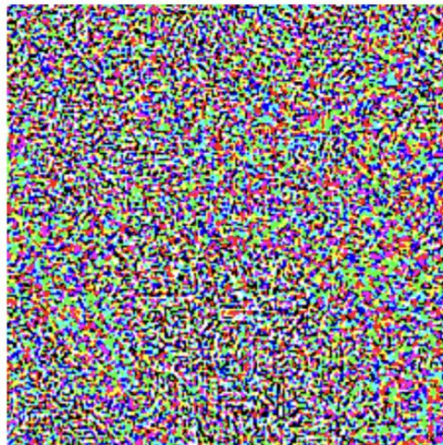


Hyperplane

But: Fooling classifiers is too easy



+ .007 ×



=



“panda”
57.7% confidence

Imperceptible
Modification

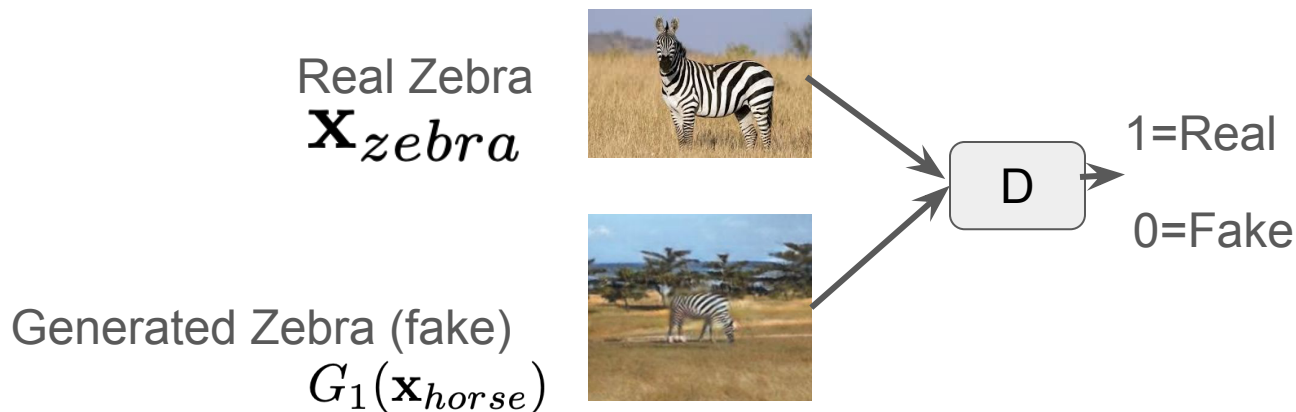
“gibbon”
99.3 % confidence

Joined training is very important!! If the discriminator D was fixed, the generator would just learn to exploit its weaknesses.

<https://www.digica.com/blog/how-to-fool-a-neural-network.html>

Train Discriminator and Generator jointly

$$\max_{G_1} \min_D [-\log D(\mathbf{x}_{zebra}) + \log (D(G_1(\mathbf{x}_{horse})))]$$



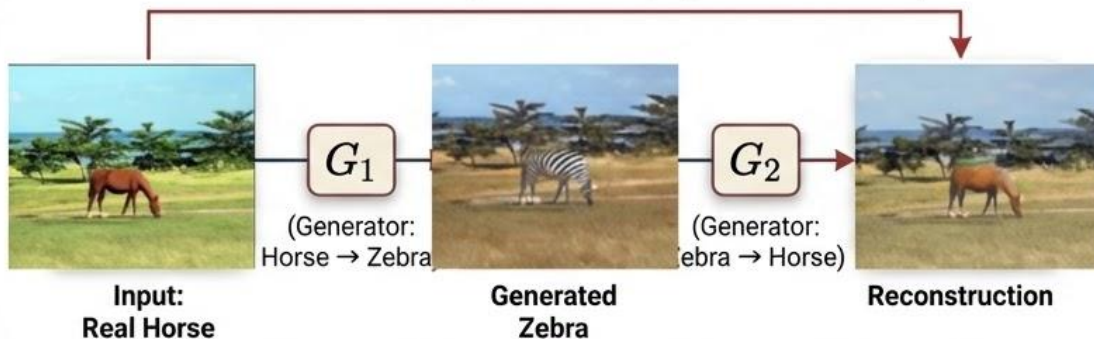


CycleGAN

Cycle Consistency & Reconstruction

$$\|\mathbf{x}_{\text{horse}} - G_2(G_1(\mathbf{x}_{\text{horse}}))\|_2^2$$

Reconstruction Loss: Squared error between original and reconstructed image, enforcing cycle consistency.



Discriminator

$$\max_{G_1} \min_D [-\log D(\mathbf{x}_{\text{zebra}}) + \log (D(G_1(\mathbf{x}_{\text{horse}})))]$$

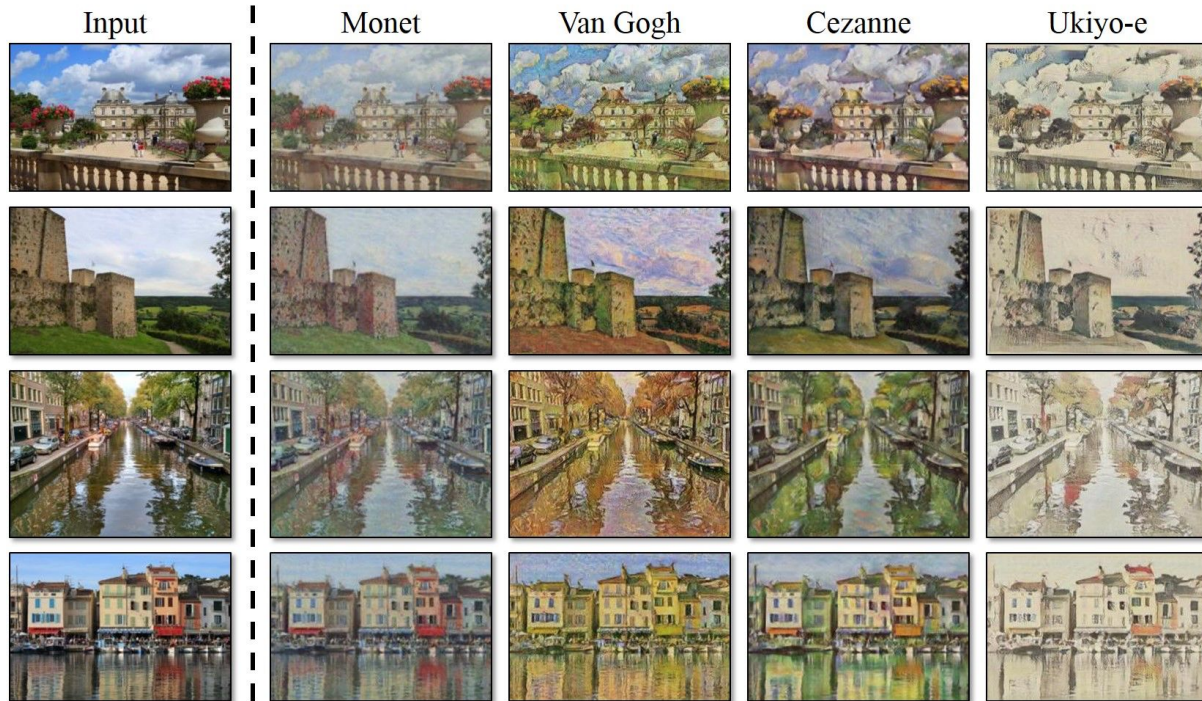


1=Real

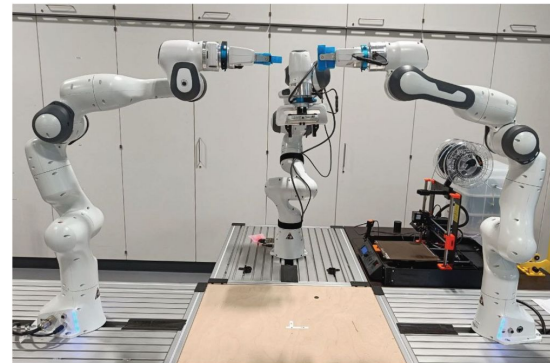
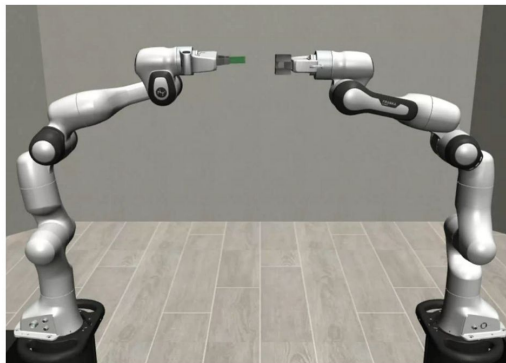
0=Fake

$$\text{Total Loss} = \text{Reconstruction Loss} + \lambda \text{ Discriminator Loss}$$

The Power of Unpaired Translation



Sim2Real Weather translation



Original

CAPIT

ForkGAN

ToDayGAN

ACL-GAN

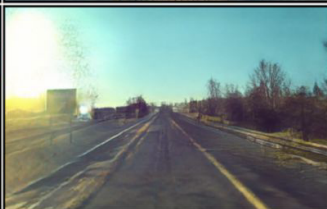
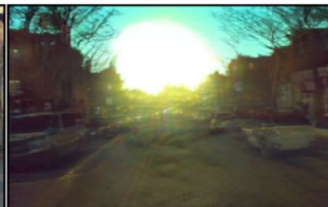
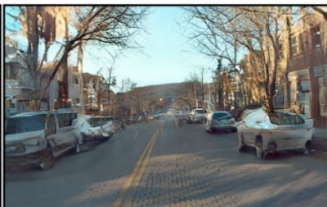
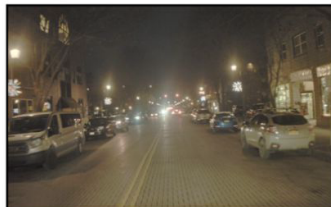


Image Transformations with Deep Learning: Four Key Cases

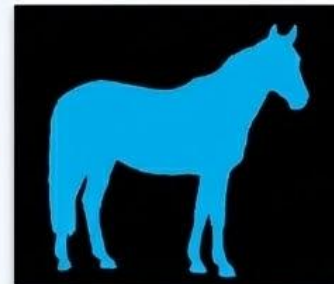
1. Image Classification



Output Label:
"Horse"

(Input: Image → Output: Label)

2. Paired Image Translation



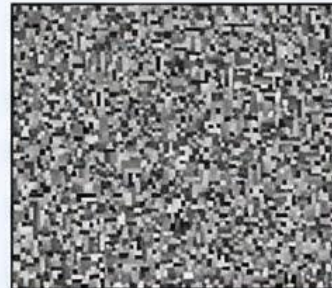
(Input: Image → Output: Image)

3. Unpaired Image Translation



(Input: Image → Output: Image)

4. Image Generation



(Input: Noise → Output: Image)

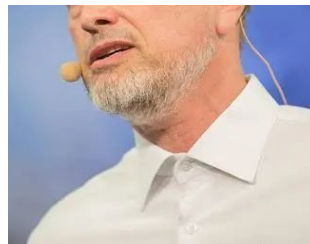
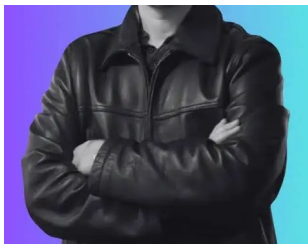
Generative Adversarial Networks are Special Cases of Artificial Curiosity (1990) and also Closely Related to Predictability Minimization (1991)

Jürgen Schmidhuber

The Swiss AI Lab, IDSIA, USI & SUPSI, Manno-Lugano

NNAISENSE, Lugano, Switzerland

Preprint (accepted by Neural Networks, Volume 127, July 2020, Pages 58-66)



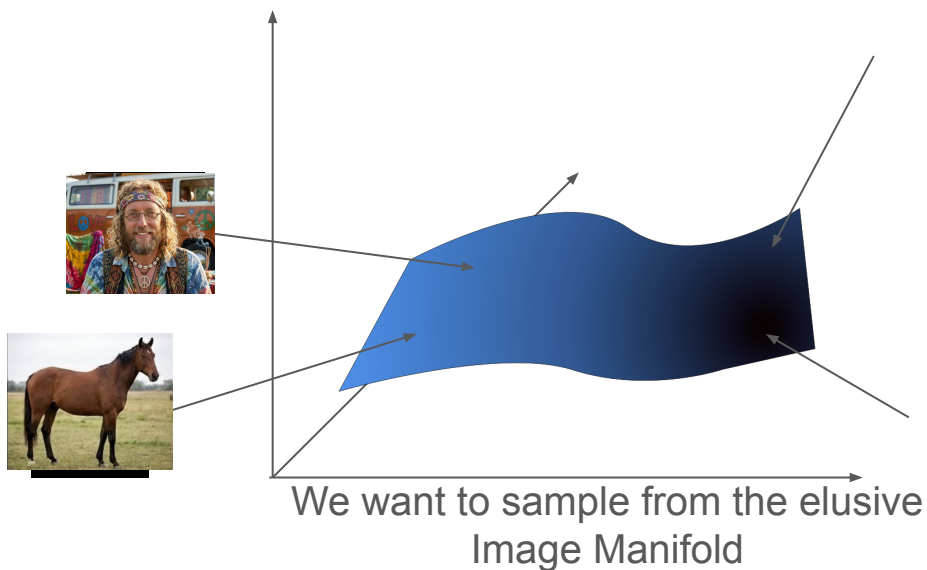
Can we directly generate something from noise?

Sample noise → sample images!

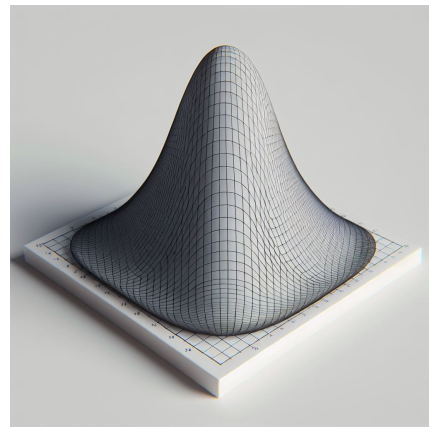


Data Manifold

- Data distribution $\mathbf{P}(\mathbf{X})$ defines a manifold of valid images
- Problem: data manifold takes up **tiny** volume of ambient space
- Naive random samples (e.g. within $[0,1]^d$) are always **off manifold**
- Solution: Sample from a Gaussian, then learn mapping to manifold



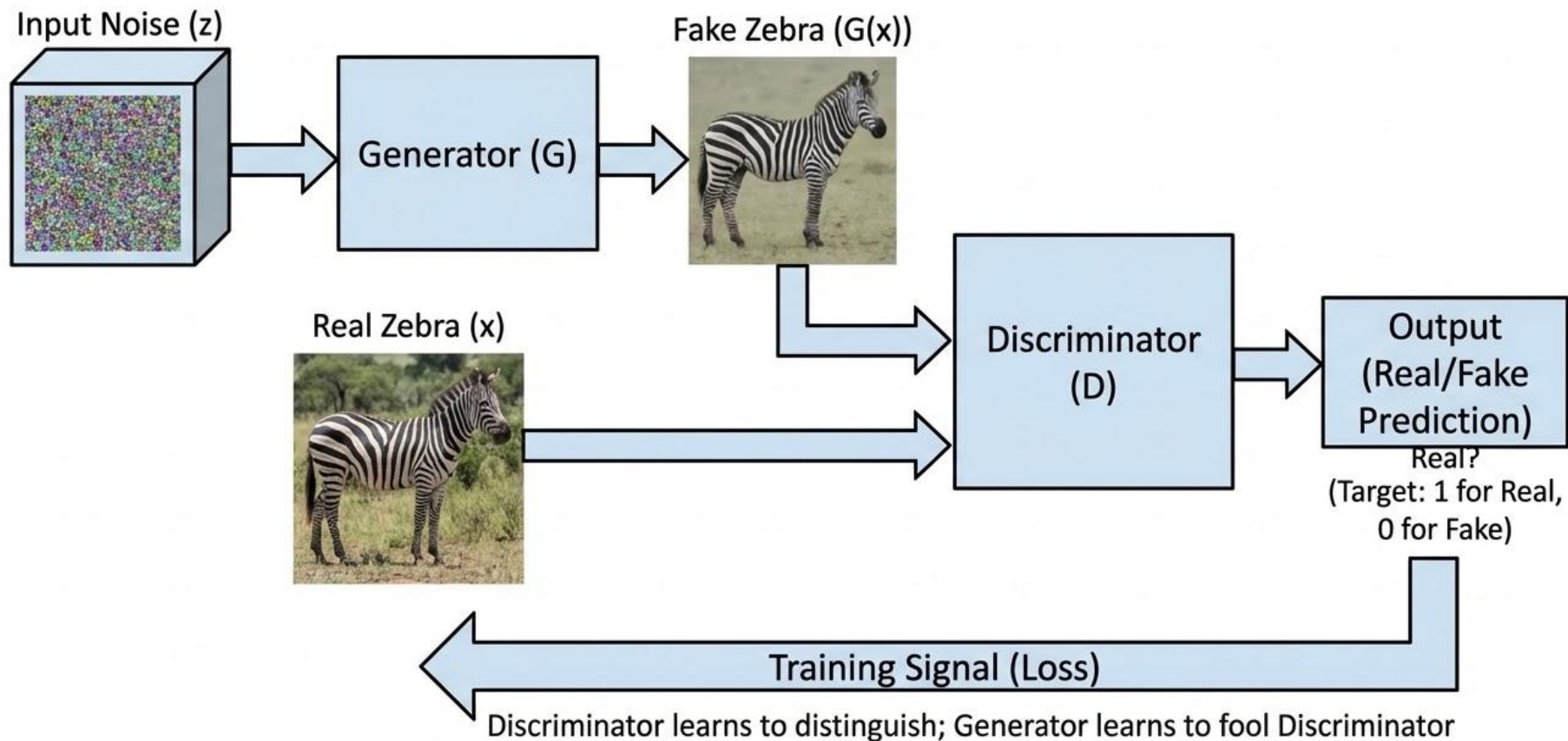
← GAN



We **can** sample from a Gaussian Distribution

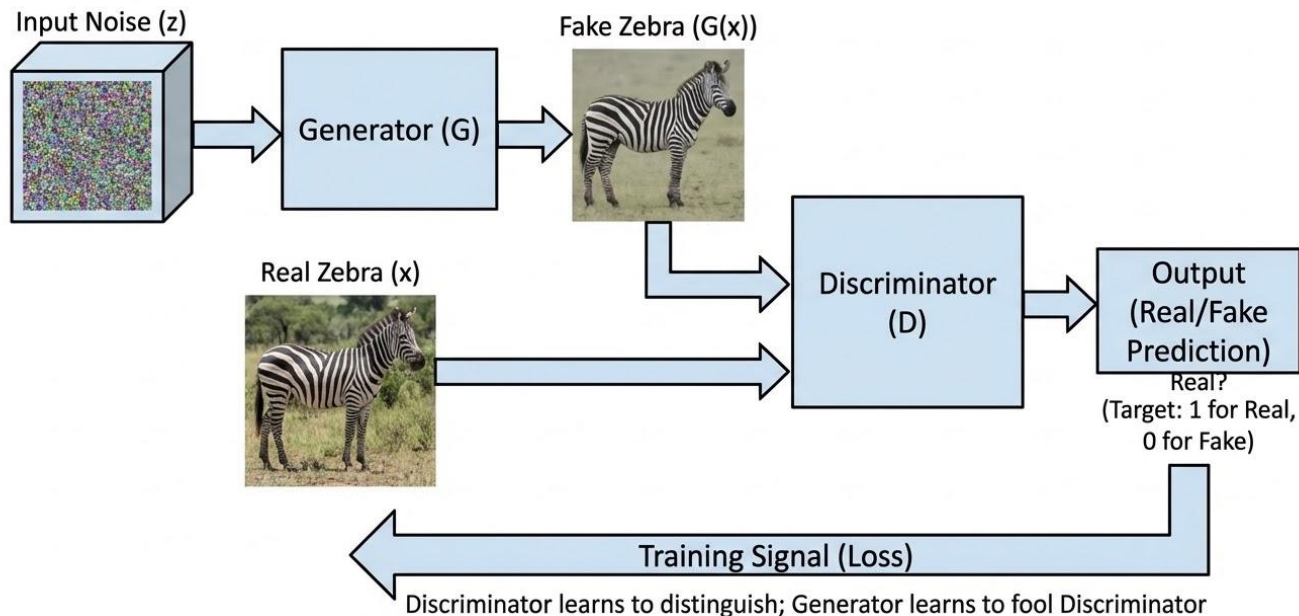


GAN Setup: Generating Zebras from Noise





GAN Setup: Generating Zebras from Noise



Discriminator:
Detect fake images

Generator:
Fool discriminator

We train the generator and discriminator jointly!

$$\max_G \min_D [-\log D(\mathbf{x}_{real}) + \log (D(G(\mathbf{z})))]$$

Example

Real dogs



dog (1)



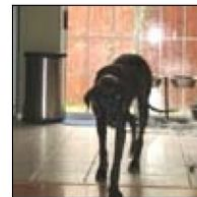
dog (1)



dog (1)

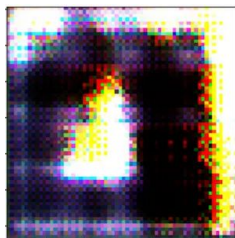


dog (1)



dog (1)

Generated



Discriminator
Loss

**Generally
Decrease**

**Ideal:
Confused**

—————▶ Training epoch

AdaIN: Conditional GAN $P(X|s)$

Adaptive Instance Normalization (AdaIN)

A normalization layer used in generative models (notably StyleGAN) to inject style information into feature maps.

The idea is simple:

1. Normalize the feature map to remove its original statistics.
2. Apply new mean and variance derived from a style vector.

$$\text{AdaIN}(x, s) = \sigma(s) \left(\frac{x - \mu(x)}{\sigma(x)} \right) + \mu(s)$$

Where:

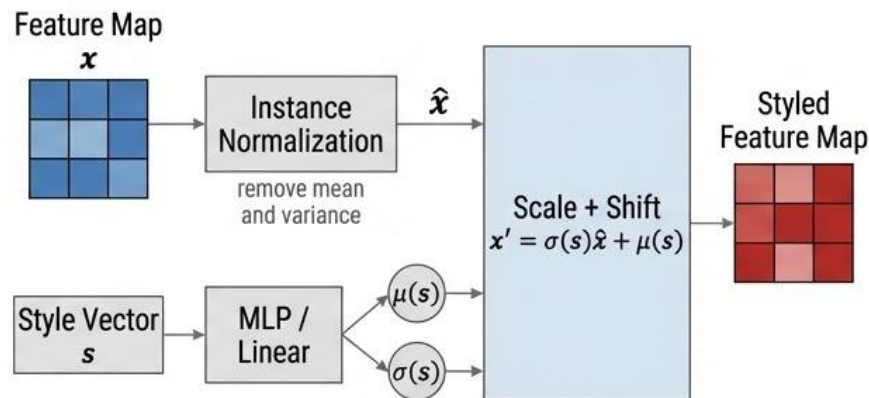
- x = feature map
- $\mu(x), \sigma(x)$ = mean and standard deviation of the feature map (per channel)
- s = style vector
- $\mu(s), \sigma(s)$ = learned scale and shift parameters produced from s

Intuition

- First remove the existing feature statistics.
- Then impose style-dependent statistics.

Thus the style vector controls how the features look (texture, colors, shapes), while the underlying features still encode structure.

Example Illustration



Visual intuition:

Feature map before AdaIN:



After AdaIN with style:

- Structure stays
- appearance statistics change

AdaIN normalizes feature maps and then re-injects **style-dependent** statistics, allowing the **generator to control appearance** through a **style vector**.

GANs

- First generative model capable of realistic high-resolution image synthesis
- Very fast generation!



Latent Space Properties

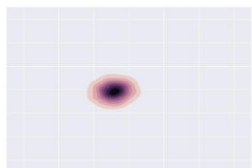
Gaussian noise vector is meaningful

- Similar noise vectors lead to similar images
- Noise dimensions are meaningful!
- Can exploit this to control generation

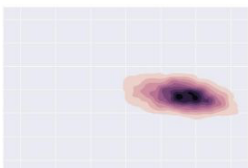


Limitations of GANs

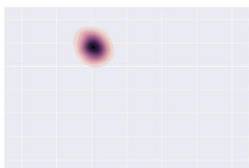
- Consider training a GAN on a dataset of dogs and cats
 - Generator could specialize in generating realistic dogs
 - Successfully fools the discriminator!
- Minimax training objective is hard to optimize!
 - Can lead to oscillations/instability during training
- Not guaranteed to converge to a good solution
 - Sensitive to hyperparameter settings, network architectures, and the choice of loss functions
- Mode Collapse is real!



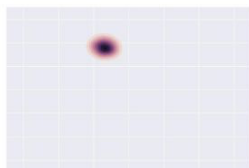
Step 0



Step 5k



Step 10k



Step 15k



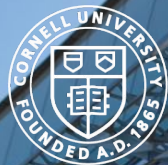
Step 20k



Step 25k



Target



Cornell Bowers C-IS

College of Computing and Information Science

Deep Learning

Week 7: The Variational Auto-Encoder (VAE)
Varsha Kishore, Justin Lovelace, Zachary Ross, Madhav
Aggarwal, Oliver Richardson

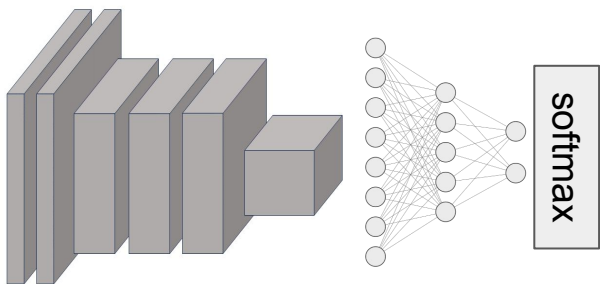
Discriminative Models

typically supervised

Goal: model $p(Y|X)$

from samples of $p(X,Y)$

(* so that we can predict
most likely labels)



Generative Models

unsupervised

Goal: model $p(X)$

from samples of $p(X)$

(* so that we can
generate artificial/new data)

Examples:

- GANs + variants
- Normalizing Flow Models
- Variational Autoencoders
 - Diffusion Models

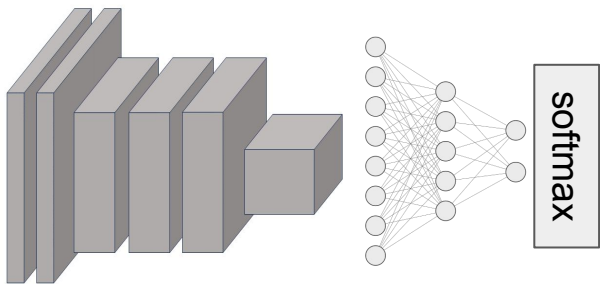
Discriminative Models

typically supervised

Goal: model $p(Y|X)$

from samples of $p(X,Y)$

(* so that we can predict
most likely labels)



Generative Models

(Conditional generation)

Goal: model $p(X|Y)$

from samples of $p(X,Y)$

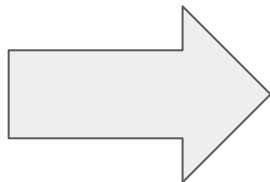
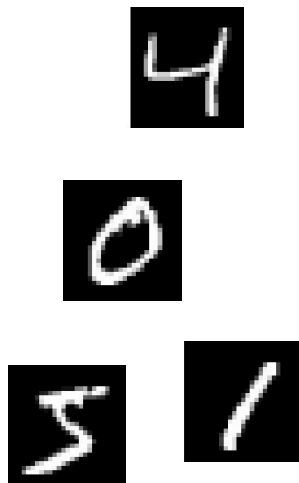
(* so that we can
generate artificial/new data)

Examples:

- GANs + variants
- Normalizing Flow Models
- Variational Autoencoders
 - Diffusion Models

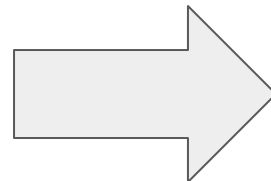
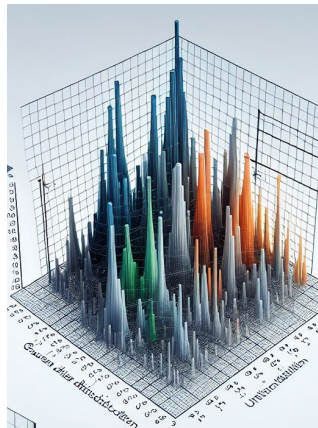
Big Picture

Data sampled from
true (but elusive) $P(X)$



Learn approximate
data distribution

$$Q(X) \approx P(X)$$

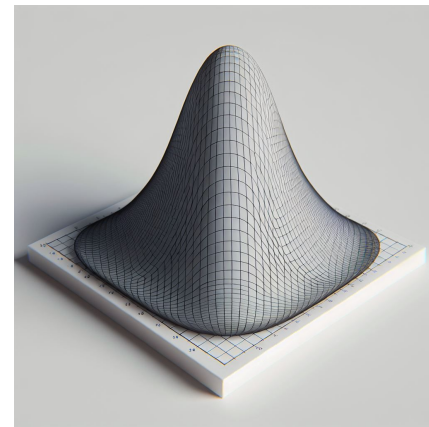
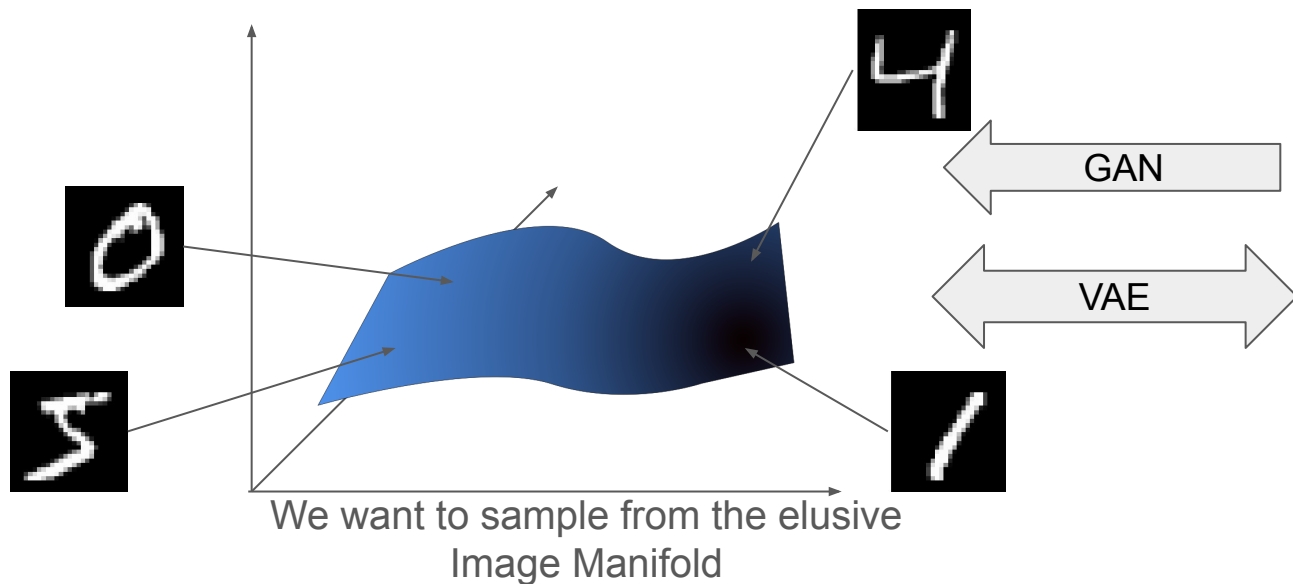


New (fake) data drawn
from $Q(x)$



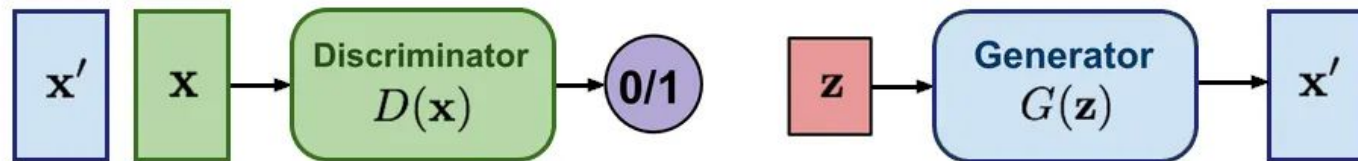
Data Manifold

- Data distribution $\mathbf{P}(\mathbf{X})$ defines a manifold of valid images
- Problem: data manifold takes up **tiny** volume of ambient space
- Naive random samples (e.g. within $[0,1]^d$) are always **off manifold**
- Solution: Sample from a Gaussian, then learn mapping to and from manifold

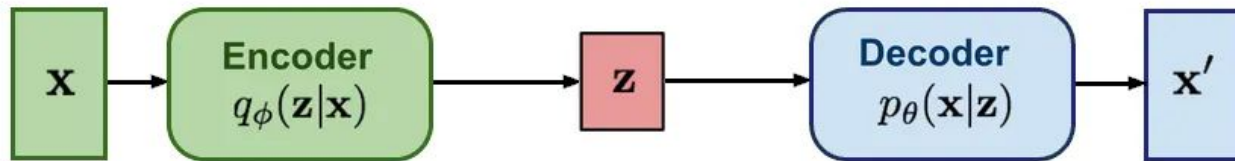


We **can** sample from a Gaussian Distribution

GAN: Adversarial training



VAE: maximize variational lower bound

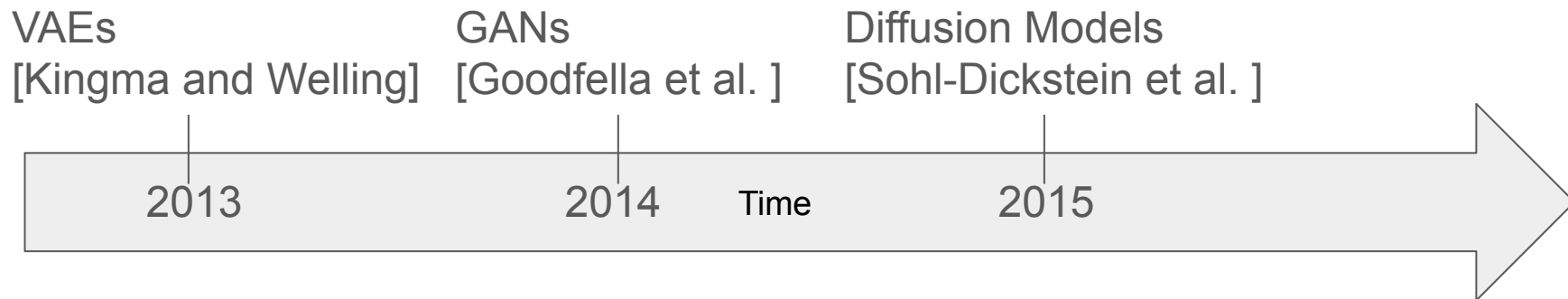


Diffusion models:
Gradually add Gaussian noise and then reverse



Timeline

- VAEs preceded GANs.
- In fact, GANs were motivated to fix some of the problems of VAEs.
- VAEs are important to understand **Diffusion Models**



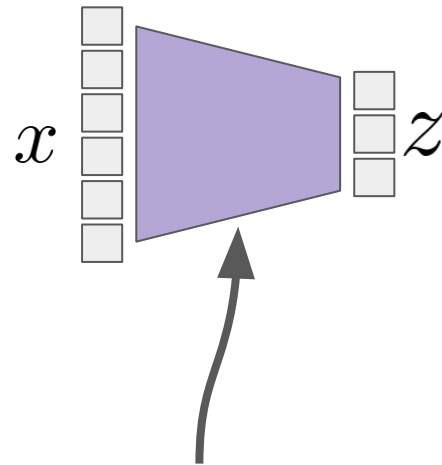
Dimensionality Reduction

Data is typically from a **low dimensional** distribution embedded in a much **higher dimensional** ambient space.

Want to map images $x \in \mathbb{R}^D$
to low-dimensional $z \in \mathbb{R}^d$

Often for the purposes of

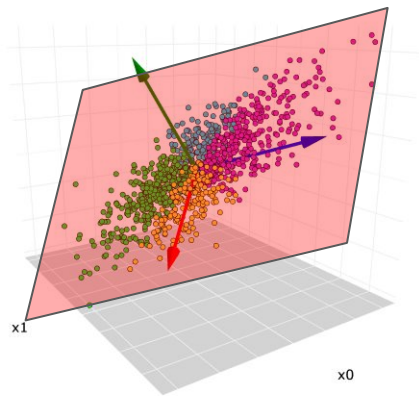
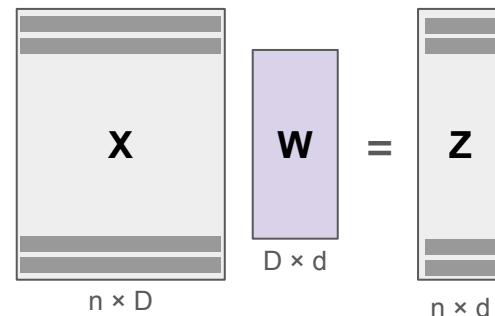
- visualization
- extracting important features
(for downstream tasks)
- representing meaningful
relationships between samples
- In this lecture sampling!



Question: what properties should this mapping have?

Principal Component Analysis (PCA)

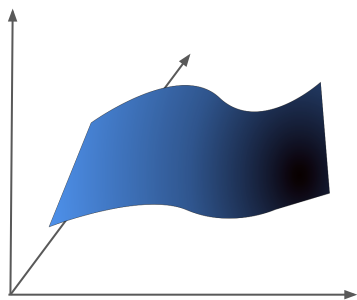
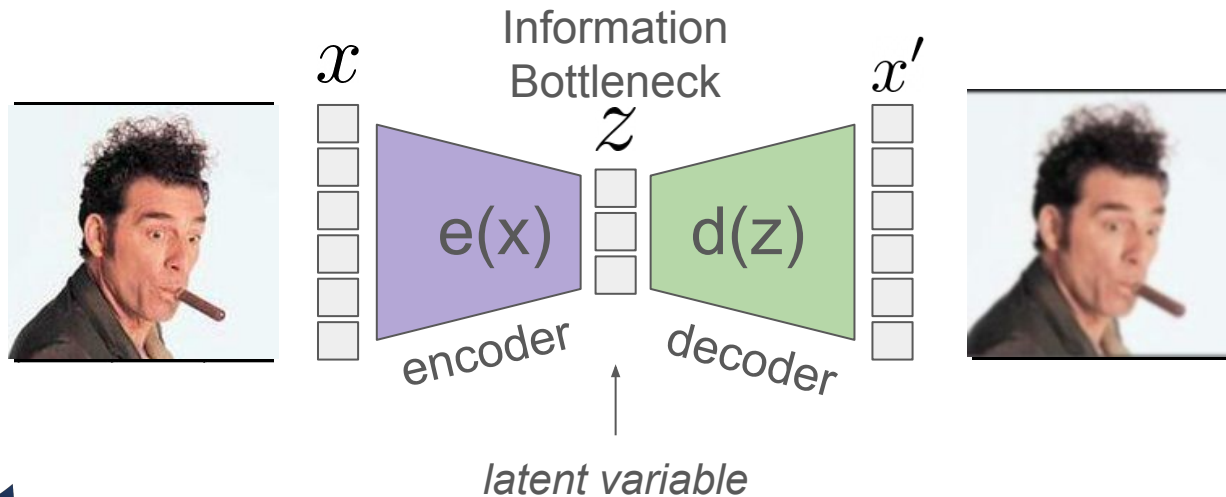
- **assumption: data manifold is a subspace**
- $\mathbf{z} = \mathbf{W}(\mathbf{x} - \mu)$ (a linear transformation)
- $\mathbf{x} \approx \mathbf{W}^\top \mathbf{z} + \mu$ (reconstruction)
- capture as much variance as possible



Can be computed directly with linear algebra:
take leading eigenvectors of (centered) scatter
matrix $\mathbf{X}\mathbf{X}'$!

Autoencoders [Kramer, 1991]

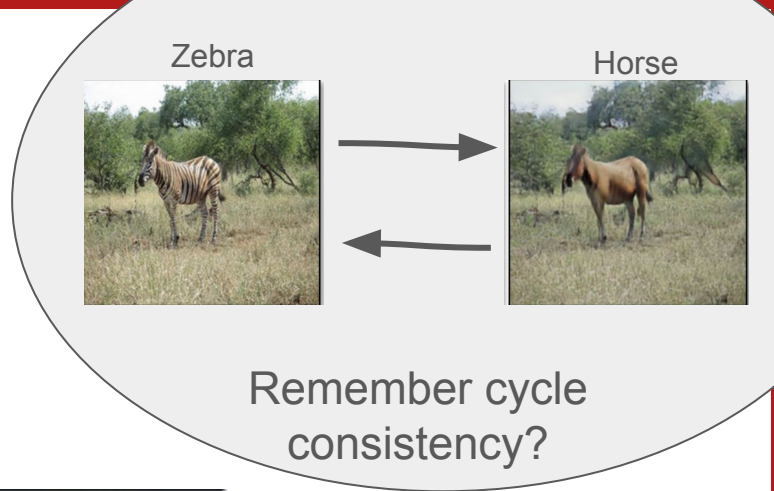
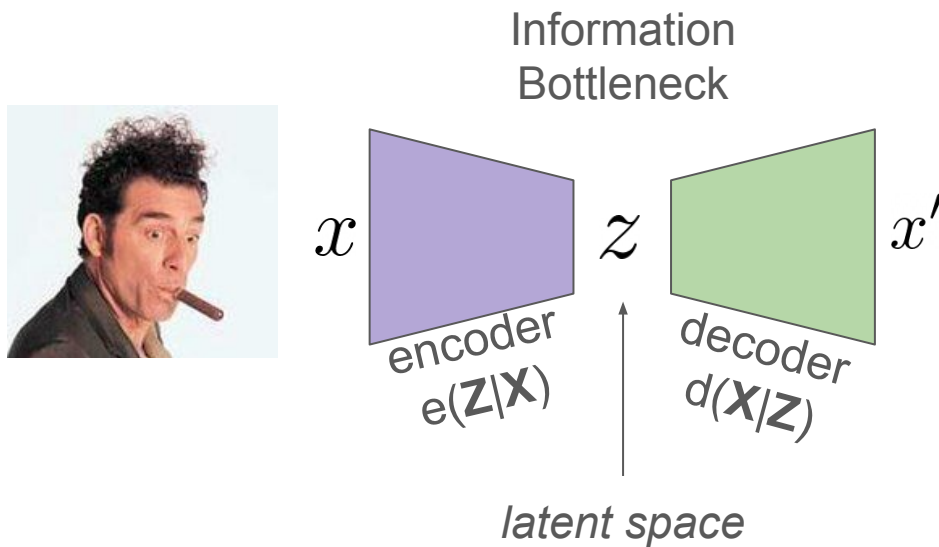
Non-linear dimensionality reduction.



Question: What loss function should we use to learn $e()$ and $d()$?
What happens if $e(x)$ and $d(z)$ are both linear functions?

Autoencoders [Kramer, 1991]

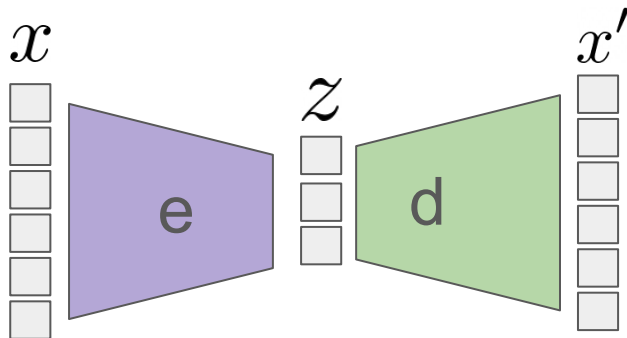
Typical loss: Squared loss, or absolute loss



$$\min_{\phi, \theta} \sum_{\mathbf{x} \in \mathcal{D}} (\mathbf{x} - \mathbf{x}')^2$$

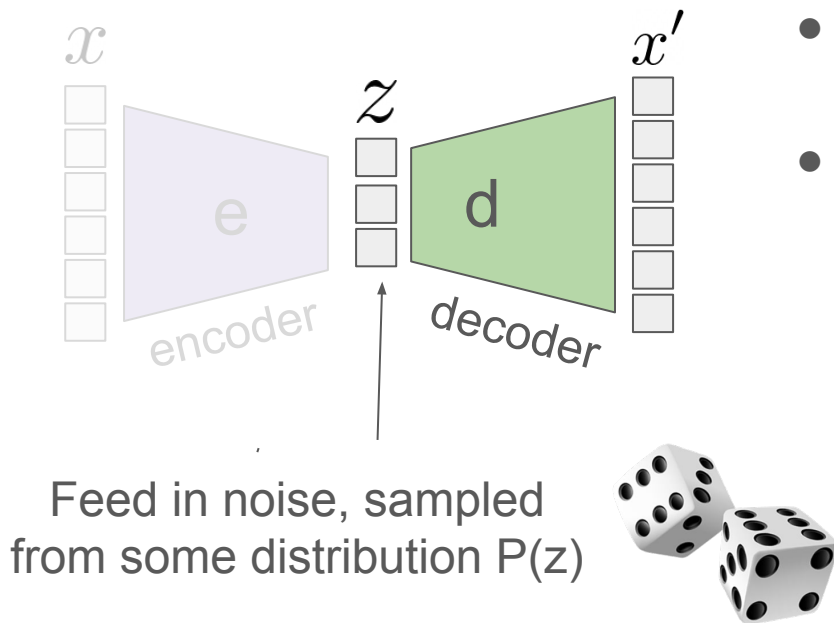
$$\mathbf{x}' = d_{\theta}(e_{\phi}(\mathbf{x}))$$

Idea: Sampling from a trained Autoencoder



- GANs train the decoder with a discriminator
- VAEs ensure quality with
 - Reconstruction loss
 - KL regularization (in a few slides)

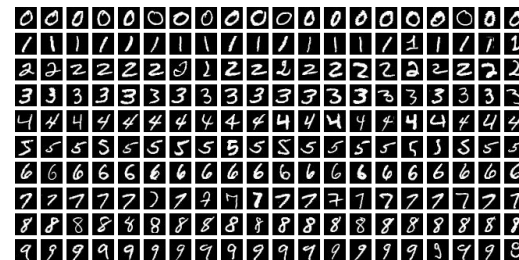
Idea: Sampling from a trained Autoencoder



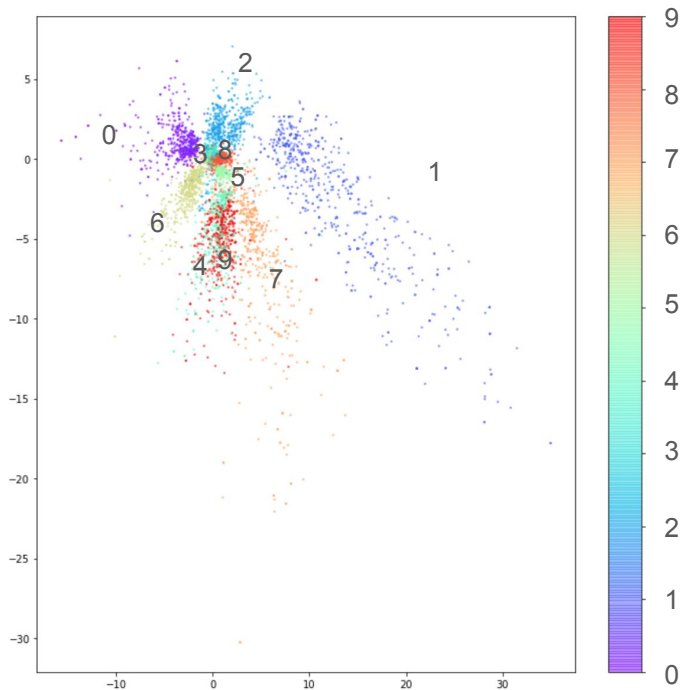
- GANs train the decoder with a discriminator
- VAEs ensure quality with
 - Reconstruction loss
 - KL regularization (in a few slides)

Crucial insight:

We can amend latent space so that it is easy to sample from it.



Autoencoder trained on MNIST: latent space



Naive representation (without any special effort), not favorable:

- lots of empty space
- no symmetries between digit representations
- **Not easy to sample in latent space**

Figure 3-8. Plot of the latent space, colored by digit

Naive sampling in latent space does not work



reconstructed sample

$$x' = d(e(x))$$



new image?

$$x' = d(\text{noise})$$

Some Fundamentals of probability and information

Building Blocks:

- Conditional and marginal probabilities
- Surprisal / Negative Log Likelihood
- Relative Entropy / KL Divergence

Conditional and Marginal Probabilities

$$p(X, Y) = p(Y|X)p(X)$$

$$p(X) = \int p(X, y) \, dy$$

Equation (2): Quick 60s Stats Puzzle

Prove that for any random variable X, Z .

$$p(\mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{z}|\mathbf{x})}$$

Hint: Later this rule will come in handy. Remember it as “Equation (2)”.

KL Divergence (a.k.a. relative entropy)

$$D(p \parallel q) := \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right]$$

reality (e.g., dataset) model

$$\mathbb{E}_{x \sim p} [\log p(x)] - \log q(x)$$

likelihood in real world; does not depend on model q Likelihood under model

- non-negative $D(p \parallel q) \geq 0$
- zero means same $D(p \parallel q) = 0 \iff p = q$
- not symmetric
- has many other, uniquely nice properties...

KL Divergence

Wei-Chiu's Coin



Kilian's Coin



$\times 2$

Question:

What is higher $KL(\text{Wei-Chiu} \parallel \text{Kilian})$ or $KL(\text{Kilian} \parallel \text{Wei-Chiu})$?

[\[link to visualization \]](#)

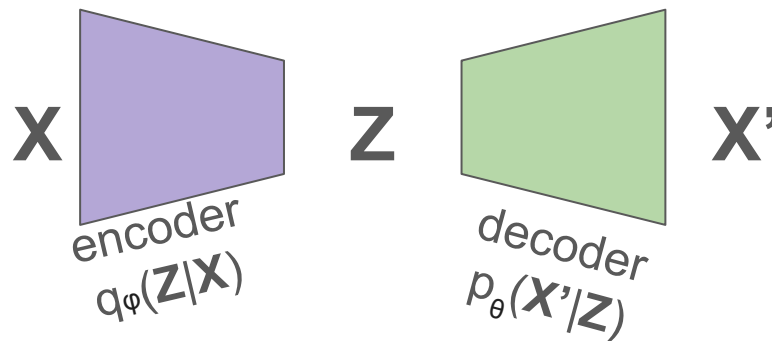
Building Blocks:

- ✓ Conditional and marginal probabilities
- ✓ Surprisal / Negative Log Likelihood
- ✓ Relative Entropy / KL Divergence

VAEs, Step 1: Make AutoEncoder probabilistic

Reconstruction Loss, using surprisal

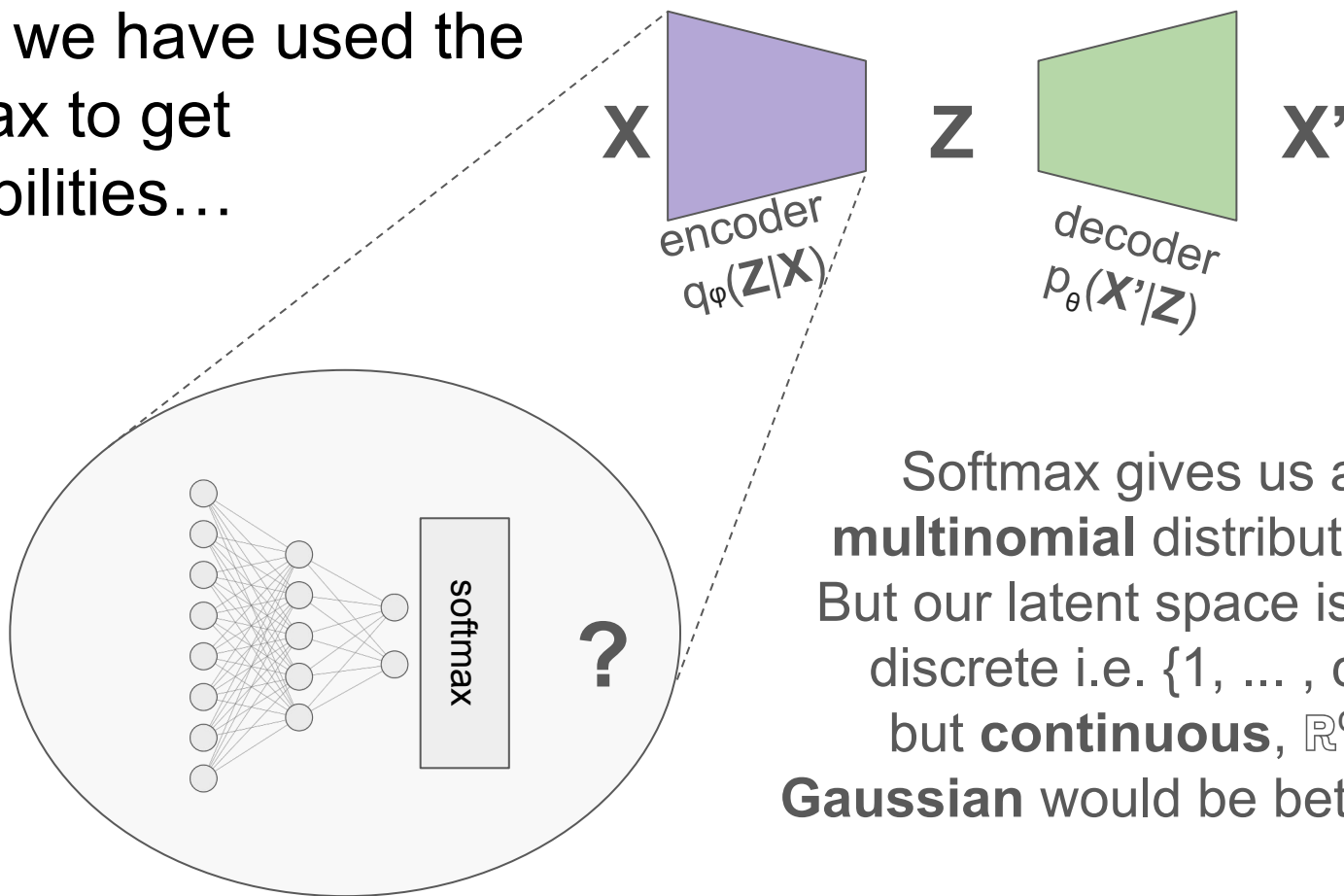
Back to our AutoEncoder,
but this time we make
everything **probabilistic**!



$$\max_{\phi, \theta} \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log (p_{\theta}(\mathbf{x}|\mathbf{z}))]$$

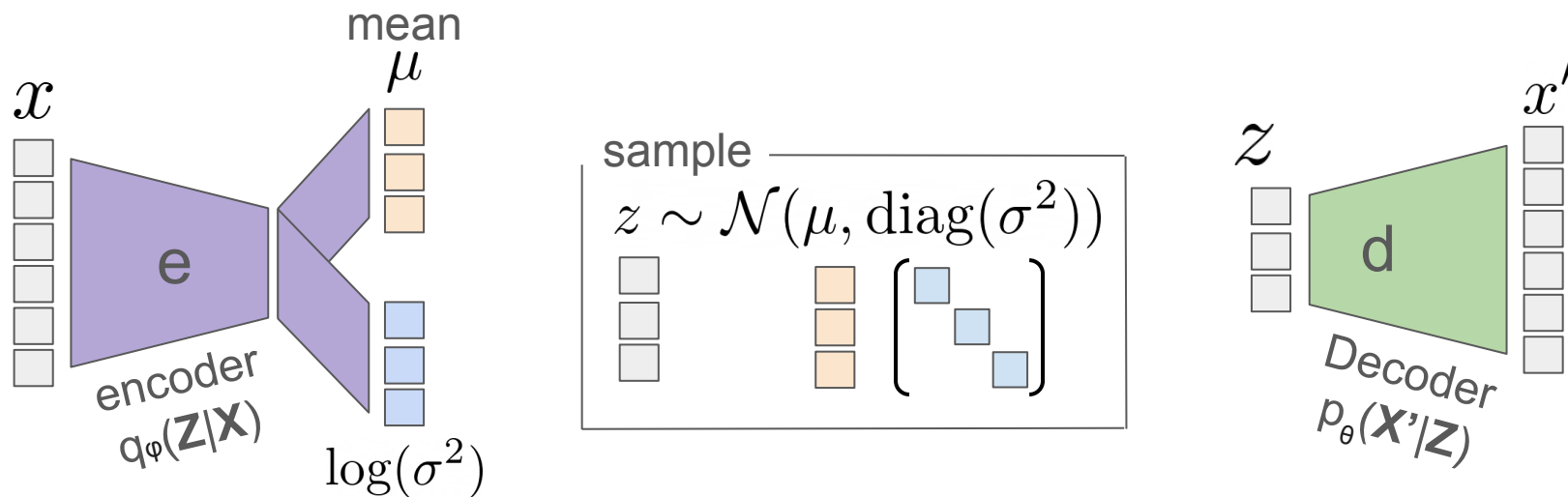
How likely would it be to encode $\mathbf{x}$,
decode the result, and recover $\mathbf{x}$?

So far we have used the softmax to get probabilities...



Softmax gives us a **multinomial** distribution. But our latent space is not discrete i.e. $\{1, \dots, c\}$, but **continuous**, $\mathbb{R}^d$! **Gaussian** would be better ...

Probabilistic **Encoder** (Gaussian)

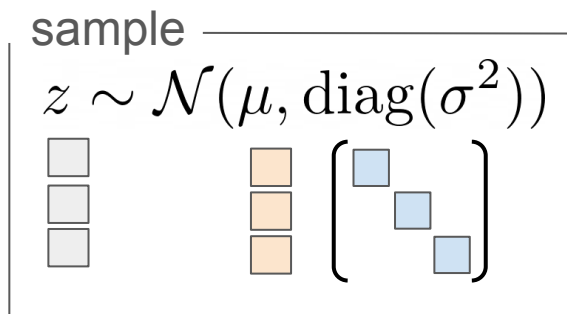


Problem: backpropagation
through sampling process?

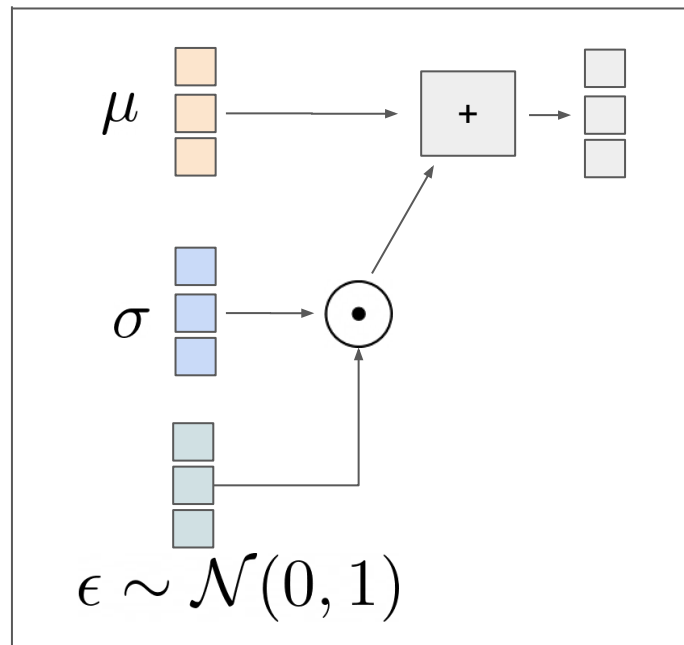
$$\max_{\phi, \theta} \mathbb{E}_{z \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log(p_\theta(\mathbf{x}|\mathbf{z}))]$$

The Reparameterization Trick

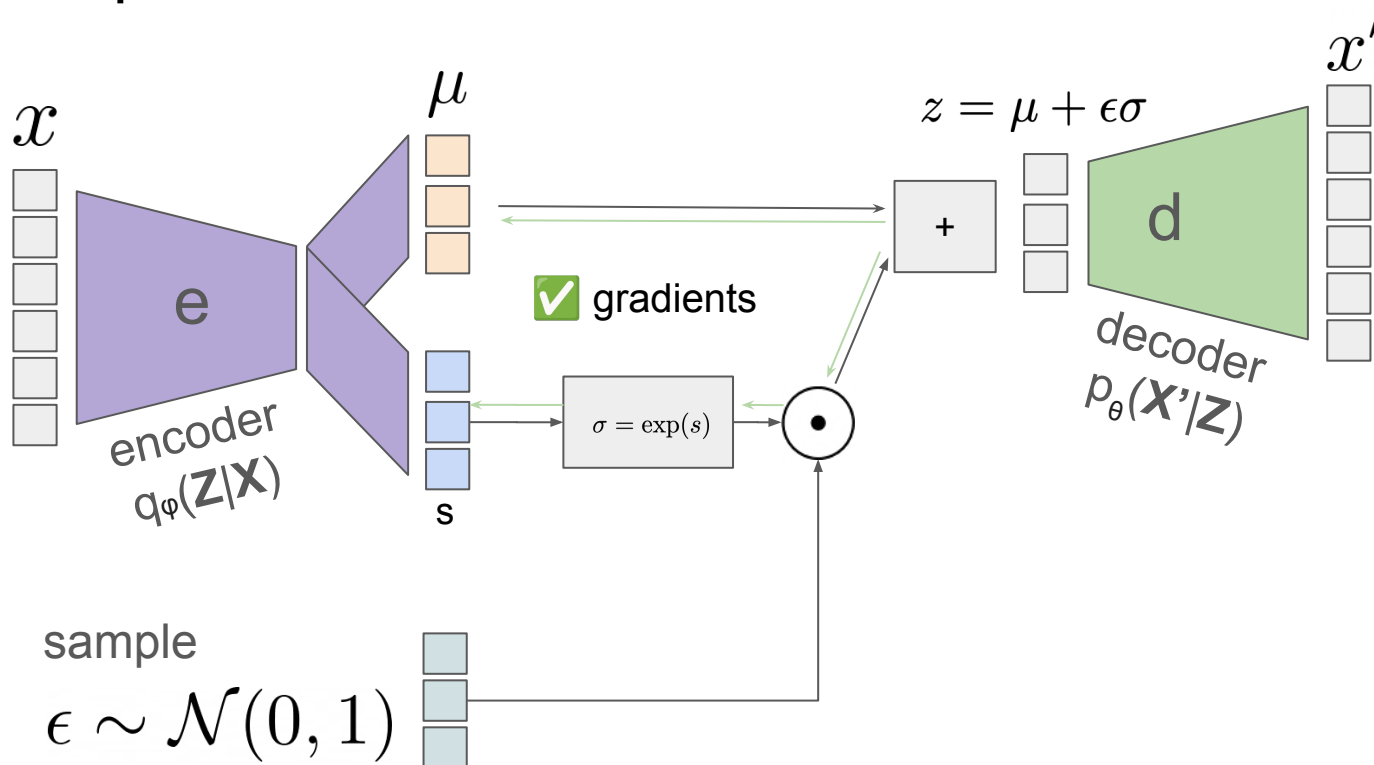
$$\mathcal{N}(\mu, \text{diag}(\sigma^2)) = \mu + \sigma \odot \mathcal{N}(0, I)$$



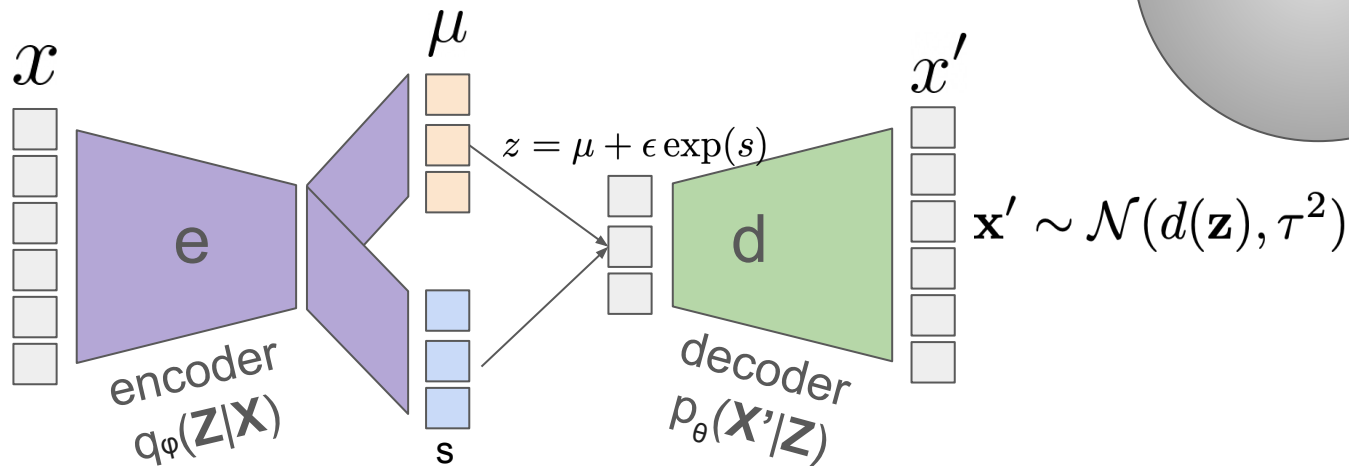
=



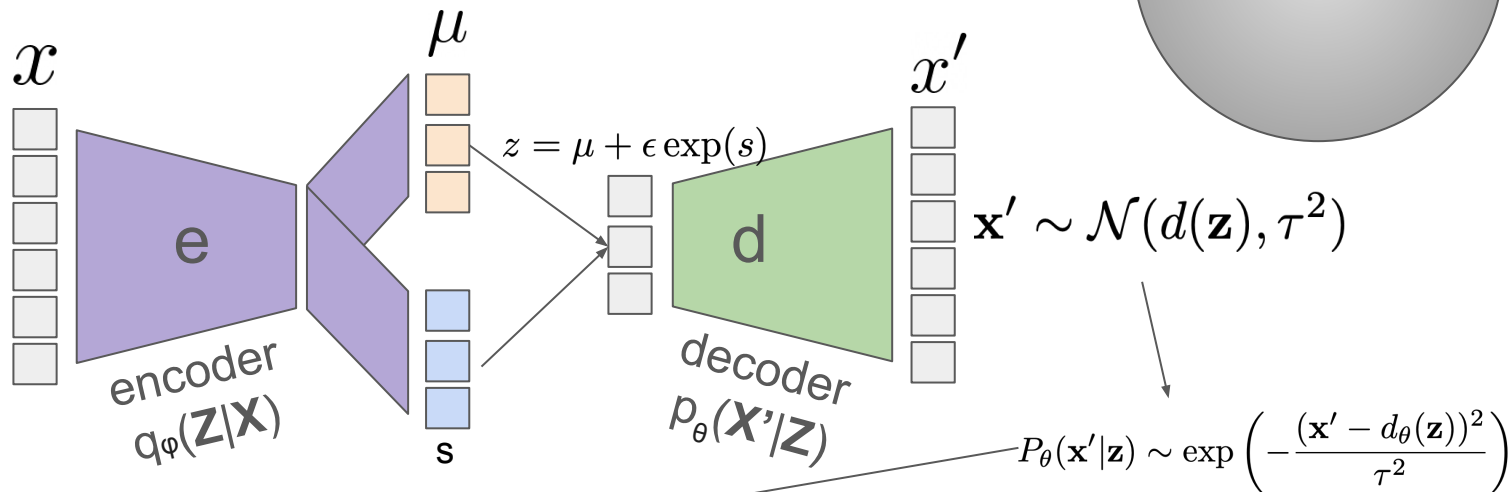
The Reparameterization Trick



Probabilistic **decoder** (Gaussian)



Probabilistic **decoder** (Gaussian)

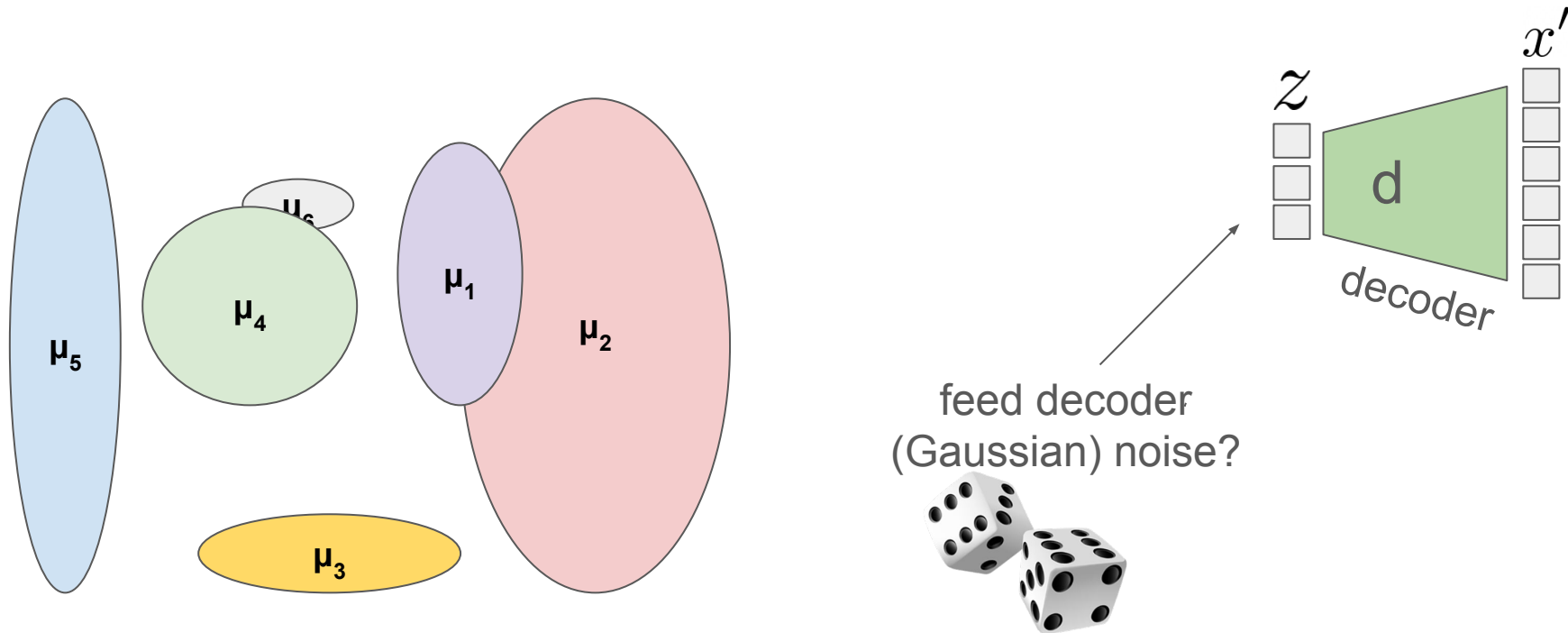


$$\max_{\phi, \theta} \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log(p_\theta(\mathbf{x}|\mathbf{z}))] = \min_{\phi, \theta} \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [(\mathbf{x} - d_\theta(\mathbf{z}))^2]$$

Plugging the output distribution into the reconstruction loss, results in the squared loss.

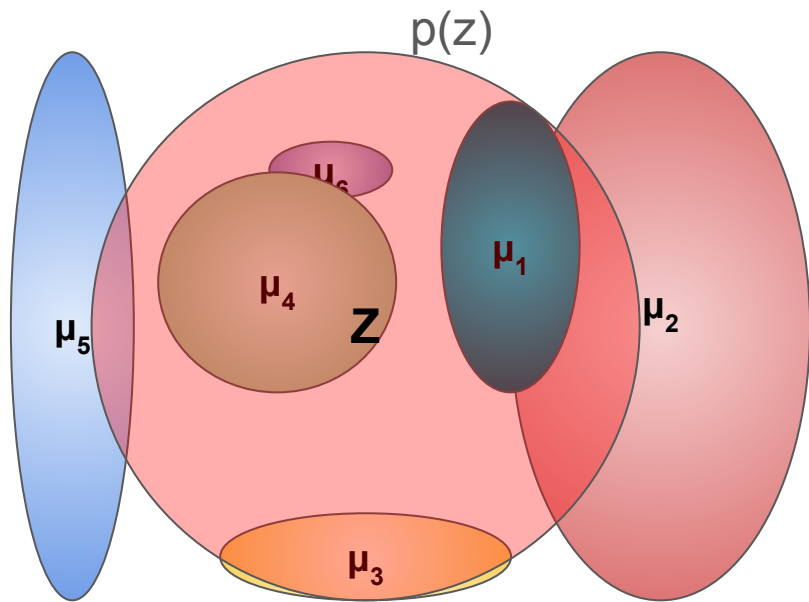
Step 2: How do we sample in latent space?

How can we sample, if each sample has its own latent distribution?



Step 2: How do we sample in latent space?

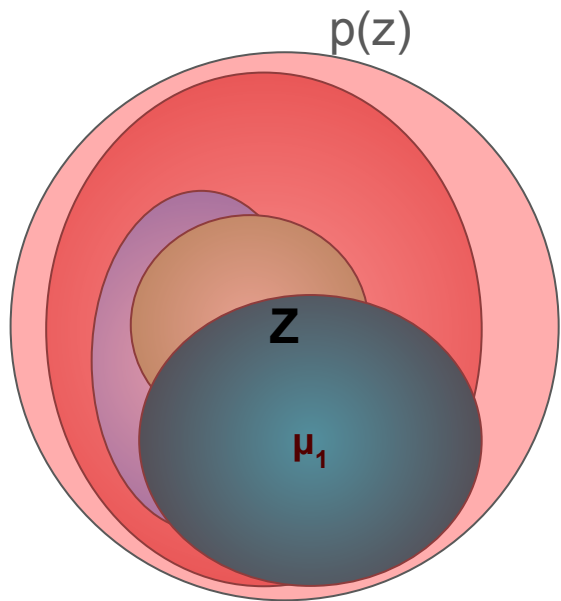
Solution: Regularize all distributions to be close to the standard normal $\mathcal{N}(\mathbf{0}; \mathbf{I})$.



$$\text{maximize} \quad \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{prior matching term}}$$

Step 2: How do we sample in latent space?

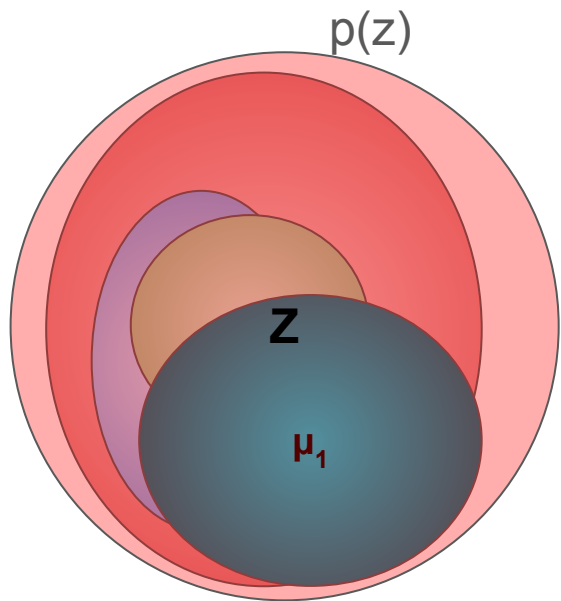
Solution: Regularize all distributions to be close to the standard normal $\mathcal{N}(\mathbf{0}; \mathbf{I})$.



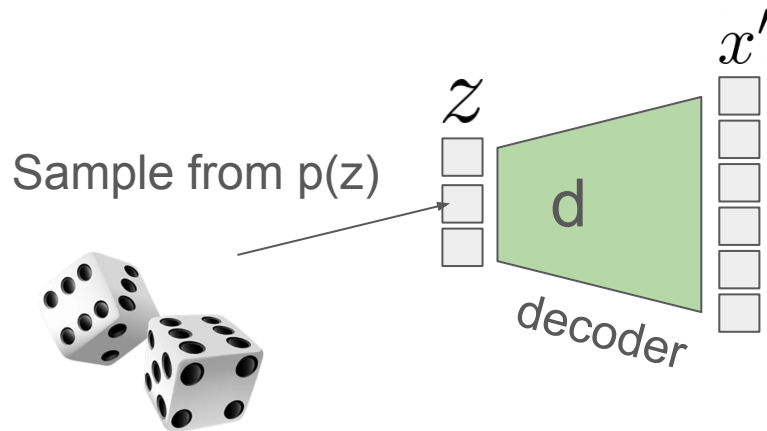
$$\text{maximize} \quad \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{prior matching term}}$$

Step 2: How do we sample in latent space?

Solution: Regularize all distributions to be close to the standard normal $\mathcal{N}(\mathbf{0}; \mathbf{I})$.



$$\text{maximize} \quad \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_\phi(z|x) \parallel p(z))}_{\text{prior matching term}}$$



Evidence Lower Bound (ELBO)

$$\log p(\mathbf{x}) = \log p(\mathbf{x}) \int q_{\phi}(\mathbf{z}|\mathbf{x}) d\mathbf{z}$$

(Multiply by $1 = \int q_{\phi}(\mathbf{z}|\mathbf{x}) d\mathbf{z}$)

$$= \int q_{\phi}(\mathbf{z}|\mathbf{x}) (\log p(\mathbf{x})) d\mathbf{z}$$

(Bring evidence into integral)

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p(\mathbf{x})]$$

(Definition of Expectation)

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{z}|\mathbf{x})} \right]$$

(Apply Equation 2)

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z}) q_{\phi}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x}) q_{\phi}(\mathbf{z}|\mathbf{x})} \right]$$

(Multiply by $1 = \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{q_{\phi}(\mathbf{z}|\mathbf{x})}$)

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] + \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right]$$

(Split the Expectation)

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] + D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}|\mathbf{x}))$$

(Definition of **KL Divergence**)

$$\geq \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right]$$

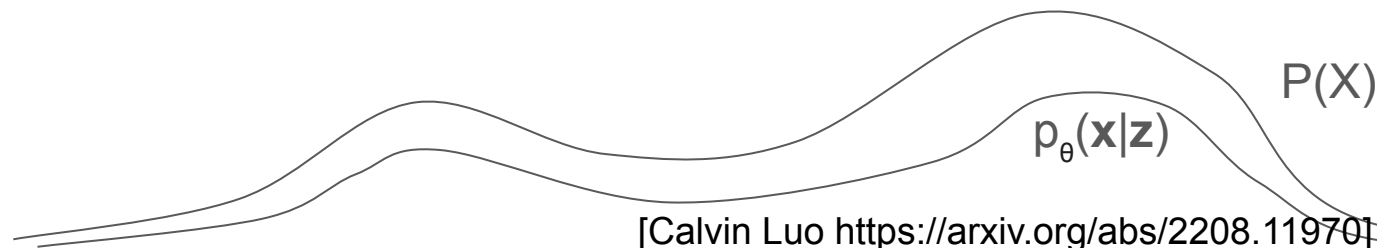
(KL Divergence always ≥ 0)

Evidence Lower Bound (ELBO)

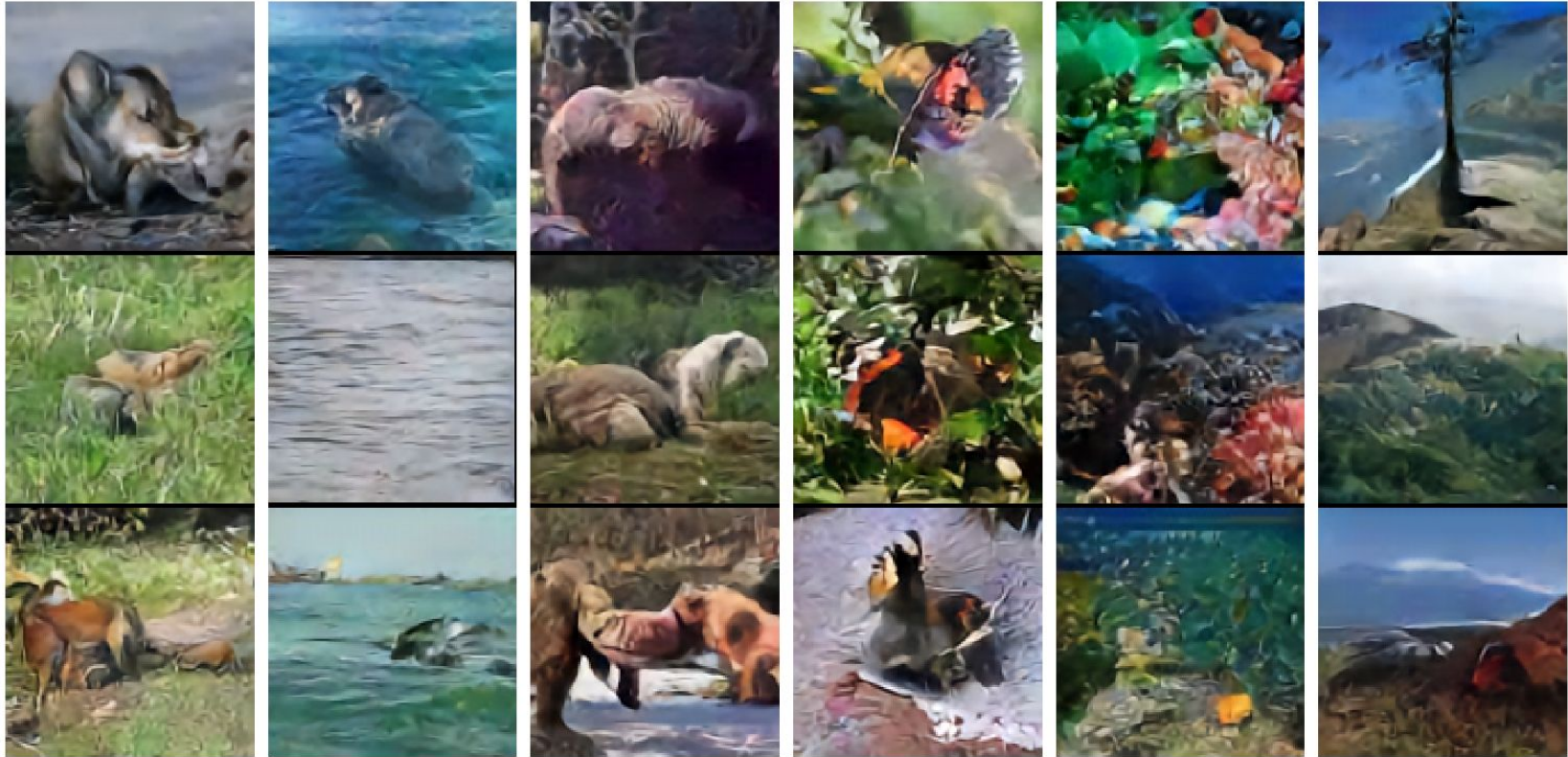
$$\begin{aligned}
 \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] && \text{(Chain Rule of Probability)} \\
 &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] + \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] && \text{(Split the Expectation)} \\
 &= \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{prior matching term}} && \text{(Definition of KL Divergence)}
 \end{aligned}$$

(We are **maximizing** this lower bound.)

If we maximize $p_{\theta}(\mathbf{x}|\mathbf{z})$ and minimize the D_{KL} we get close to $P(\mathbf{x})$.



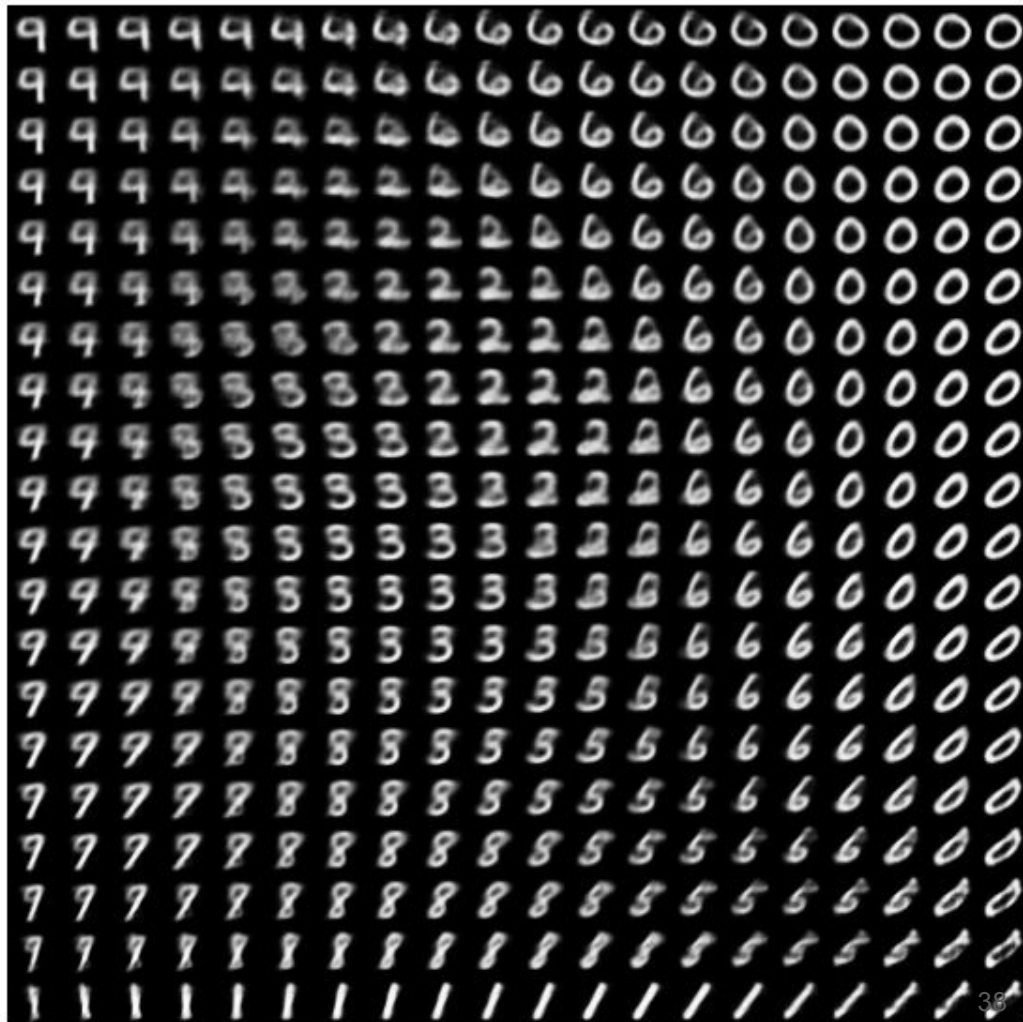
Examples of VAE generated images



a much nicer space...

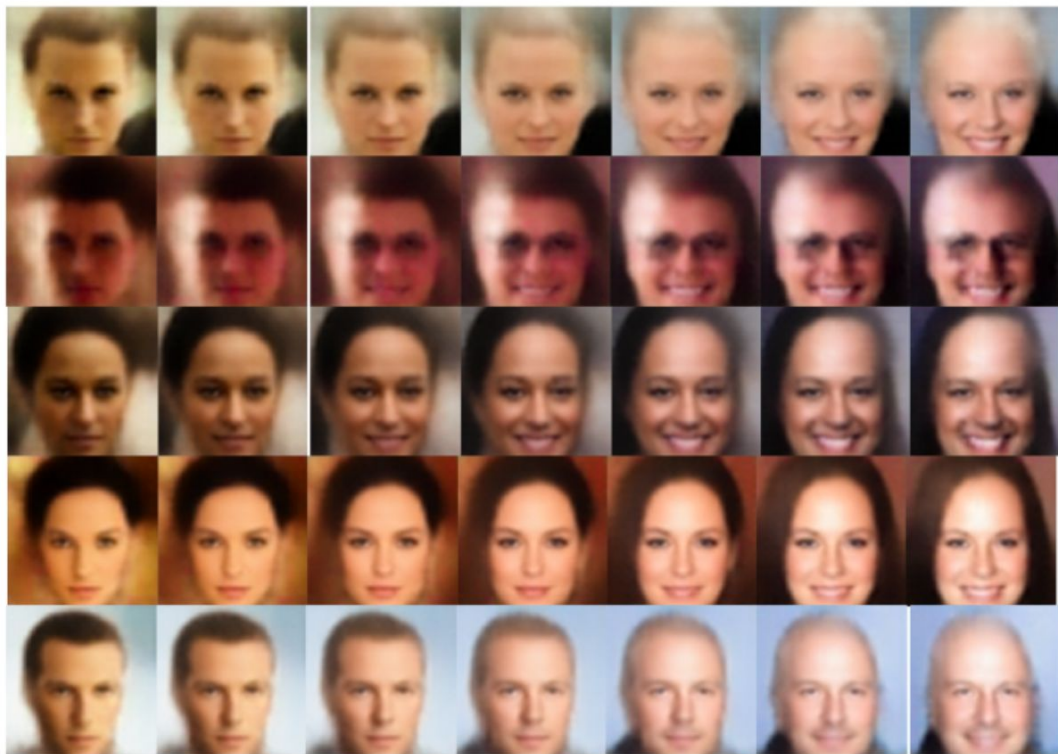
can smoothly interpolate digits in
a meaningful, digit-y kind of way

[DEMO](#)



a much nicer space

dimensions in latent space correspond to meaningful concepts, like sentiment and orientation



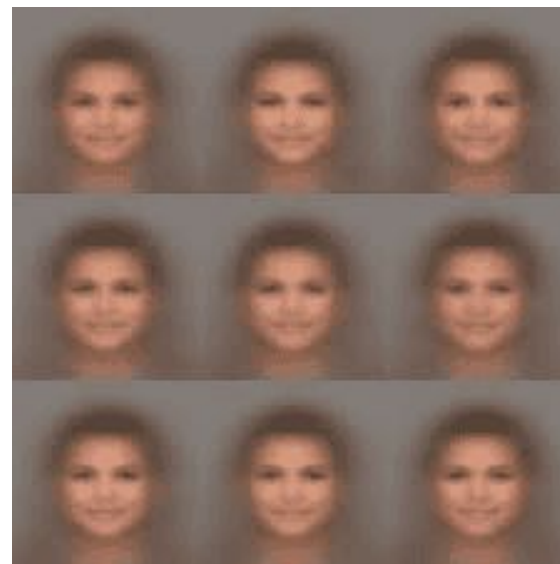
The Biggest Drawback of VAEs

- Out of the box, generated images can be blurry.

Question: Why? How do GANs fix this problem?

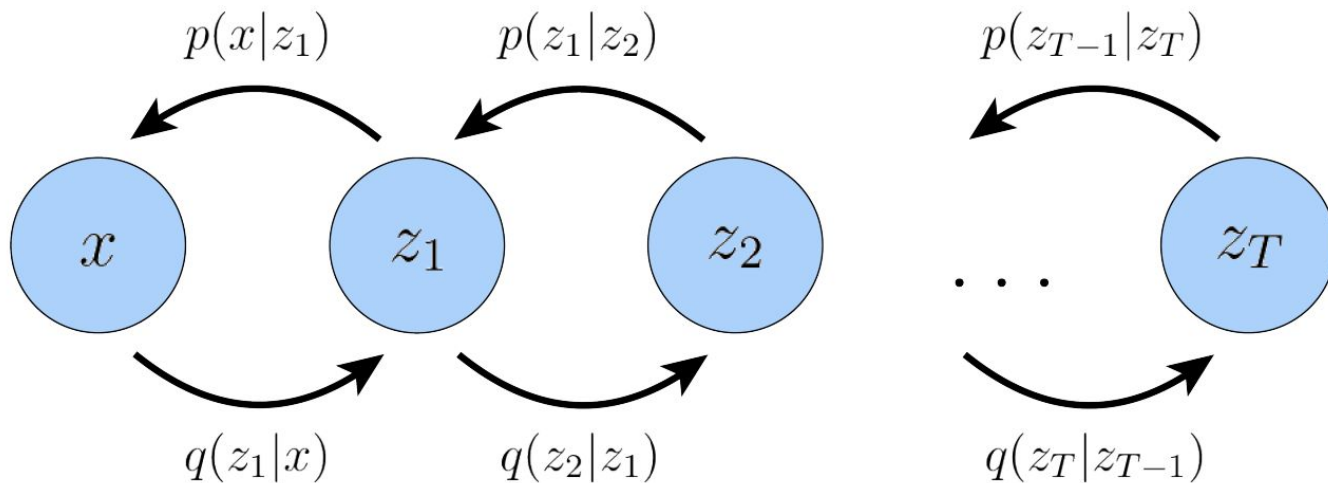


[VAE v. GAN](#)



Hierarchical VAEs

The generative process is modeled as a Markov chain, where each latent z_t is generated only from the previous latent z_{t+1}



Summary

- Generative Image models learn a mapping from the **Standard Normal Gaussian** to the **Image Manifold**
 - GANs learn this through a **discriminator**.
 - VAEs learn it through **variational autoencoders**
- AutoEncoders learn to **compress** and **reconstruct** data
- VAEs make these AutoEncoders **probabilistic**
 - Minimize the **reconstruction loss**
 - Latent space is sampled from Gaussian distributions
 - Sampling is made differentiable with the **Reparameterization Trick**
 - Deviations from the Prior (Standard Normal Gaussian) is penalized by **KL divergence**
- The ELBO is a **lower bound** of $P(X)$
 - Maximizing the ELBO, and minimizing the KL divergence makes $P(x|z)$ close to $P(x)$